SolidityScan | Powered by CRED SHIELDS

Security Assessment Report

XOX Staking

24 Feb 2026

This security assessment report was prepared by SolidityScan.com, a cloud-based Smart Contract Scanner.

Table of Contents

01 Vulnerability Classification and Severity

02 Executive Summary

03 Findings Summary

04 Vulnerability Details

PRECISION LOSS DURING DIVISION BY LARGE NUMBERS

LACK OF ZERO VALUE CHECK IN TOKEN TRANSFERS

MISSING EVENTS

OUTDATED COMPILER VERSION

ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT

BLOCK VALUES AS A PROXY FOR TIME

CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES

CONTRACT NAME SHOULD USE PASCALCASE

MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS

MISSING @INHERITDOC ON OVERRIDE FUNCTIONS

MISSING NATSPEC COMMENTS IN SCOPE BLOCKS

MISSING NATSPEC DESCRIPTIONS FOR PUBLIC VARIABLE DECLARATIONS

MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS

MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS

MISSING UNDERSCORE IN NAMING VARIABLES

NAME MAPPING PARAMETERS

REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS
ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT
AVOID RE-STORING VALUES
AVOID ZERO-TO-ONE STORAGE WRITES
CACHE ADDRESS(THIS) WHEN USED MORE THAN ONCE
CHEAPER INEQUALITIES IN IF()
DEFAULT INT VALUES ARE MANUALLY RESET
DEFINE CONSTRUCTOR AS PAYABLE
FUNCTIONS CAN BE IN-LINED
REVERTING FUNCTIONS CAN BE PAYABLE
FUNCTION SHOULD RETURN STRUCT
GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION
SMALLER DATA TYPES COST MORE
STORAGE VARIABLE CACHING IN MEMORY
STORING STORAGE VARIABLES IN MEMORY

05 Scan History

06 Disclaimer

01. Vulnerability Classification and Severity

Description

To enhance navigability, the document is organized in descending order of severity for easy reference. Issues are categorized as *Fixed*, *Pending Fix*, or *Won't Fix*, indicating their current status. *Won't Fix* denotes that the team is aware of the issue but has chosen not to resolve it. Issues labeled as *Pending Fix* state that the bug is yet to be resolved. Additionally, each issue's severity is assessed based on the risk of exploitation or the potential for other unexpected or unsafe behavior.

Critical

The issue affects the contract in such a way that funds may be lost, allocated incorrectly, or otherwise result in a significant loss.

High

High-severity vulnerabilities pose a significant risk to both the Smart Contract and the organization. They can lead to user fund losses, may have conditional requirements, and are challenging to exploit.

Medium

The issue affects the ability of the contract to operate in a way that doesn't significantly hinder its behavior.

Low

The issue has minimal impact on the contract's ability to operate.

Informational

The issue does not affect the contract's operational capability but is considered good practice to address.

Gas

This category deals with optimizing code and refactoring to conserve gas.

02. Executive Summary

XOX Staking

Uploaded Solidity File(s)

Language	Audit Methodology	Website
Solidity	**Static Scanning**	-
Publishers/Owner Name	Organization	Contact Email
-	-	-

Security Score is GREAT

The SolidityScan score is calculated based on lines of code and weights assigned to each issue depending on the severity and confidence. To improve your score, view the detailed result and leverage the remediation solutions provided.

This report has been prepared for XOX Staking using SolidityScan to scan and discover vulnerabilities and safe coding practices in their smart contract including the libraries used by the contract that are not officially recognized. The SolidityScan tool runs a comprehensive static analysis on the Solidity code and finds vulnerabilities ranging from minor gas optimizations to major vulnerabilities leading to the loss of funds. The coverage scope pays attention to all the informational and critical vulnerabilities with over 700+ modules. The scanning and auditing process covers the following areas:

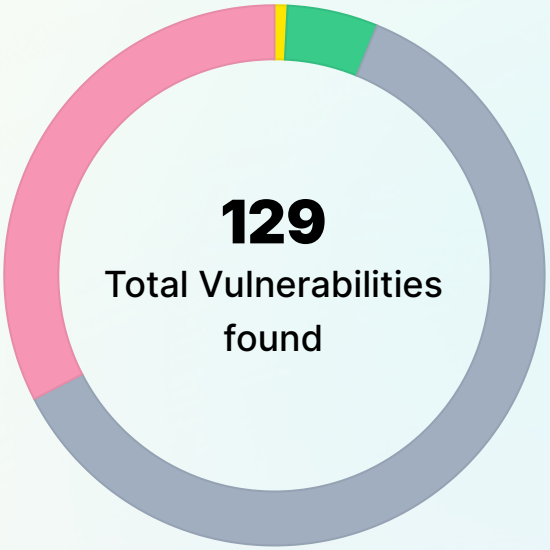
Various common and uncommon attack vectors will be investigated to ensure that the smart contracts are secure from malicious actors. The scanner modules find and flag issues related to Gas optimizations that help in reducing the overall Gas cost It scans and evaluates the codebase against industry best practices and standards to ensure compliance It makes sure that the officially recognized libraries used in the code are secure and up to date.

The SolidityScan Team recommends running regular audit scans to identify any vulnerabilities that are introduced after XOX Staking introduces new features or refactors the code.

03. Findings Summary

XOX Staking
File Scan

Security Score	Scan duration	Lines of code
93.37/100	28 secs	400



0	0	1	7	79	42
Crit	High	Med	Low	Info	Gas

This audit report has not been verified by the SolidityScan team. To learn more about our published reports. click here

ACTION TAKEN

0	0	0	133
Fixed	*False Positive*	*Won't Fix*	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
M001	Medium	PRECISION LOSS DURING DIVISION BY LARGE NUMBERS	1	Automated	*Pending Fix*
L001	Low	LACK OF ZERO VALUE CHECK IN TOKEN TRANSFERS	4	Automated	*Pending Fix*
L002	Low	MISSING EVENTS	3	Automated	*Pending Fix*
L003	Low	OUTDATED COMPILER VERSION	1	Automated	*Pending Fix*
I001	Informational	ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT	1	Automated	*Pending Fix*
I002	Informational	BLOCK VALUES AS A PROXY FOR TIME	3	Automated	*Pending Fix*
I003	Informational	CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES	3	Automated	*Pending Fix*
I004	Informational	CONTRACT NAME SHOULD USE PASCALCASE	1	Automated	*Pending Fix*
I005	Informational	MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	*Pending Fix*
I006	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	*Pending Fix*
I007	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS	10	Automated	*Pending Fix*
I008	Informational	MISSING @INHERITDOC ON OVERRIDE FUNCTIONS	2	Automated	*Pending Fix*
I009	Informational	MISSING NATSPEC COMMENTS IN SCOPE BLOCKS	8	Automated	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
I010	Informational	MISSING NATSPEC DESCRIPTIONS FOR PUBLIC VARIABLE DECLARATIONS	11	Automated	*Pending Fix*
I011	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS	1	Automated	*Pending Fix*
I012	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS	1	Automated	*Pending Fix*
I013	Informational	MISSING UNDERSCORE IN NAMING VARIABLES	16	Automated	*Pending Fix*
I014	Informational	NAME MAPPING PARAMETERS	1	Automated	*Pending Fix*
I015	Informational	REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS	25	Automated	*Pending Fix*
G001	Gas	ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT	4	Automated	*Pending Fix*
G002	Gas	AVOID RE-STORING VALUES	1	Automated	*Pending Fix*
G003	Gas	AVOID ZERO-TO-ONE STORAGE WRITES	1	Automated	*Pending Fix*
G004	Gas	CACHE ADDRESS(THIS) WHEN USED MORE THAN ONCE	9	Automated	*Pending Fix*
G005	Gas	CHEAPER INEQUALITIES IN IF()	8	Automated	*Pending Fix*
G006	Gas	DEFAULT INT VALUES ARE MANUALLY RESET	5	Automated	*Pending Fix*
G007	Gas	DEFINE CONSTRUCTOR AS PAYABLE	1	Automated	*Pending Fix*
G008	Gas	FUNCTIONS CAN BE IN-LINED	1	Automated	*Pending Fix*
G009	Gas	REVERTING FUNCTIONS CAN BE PAYABLE	5	Automated	*Pending Fix*
G010	Gas	FUNCTION SHOULD RETURN STRUCT	1	Automated	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
G011	Gas	GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION	1	Automated	*Pending Fix*
G012	Gas	SMALLER DATA TYPES COST MORE	1	Automated	*Pending Fix*
G013	Gas	STORAGE VARIABLE CACHING IN MEMORY	10	Automated	*Pending Fix*
G014	Gas	STORING STORAGE VARIABLES IN MEMORY	5	Automated	*Pending Fix*

04. Vulnerability Details

Issue Type

PRECISION LOSS DURING DIVISION BY LARGE NUMBERS

S. No.	Severity	Detection Method	Instances
M001	Medium	Automated	1

Description

In Solidity, when dividing large numbers, precision loss can occur due to limitations in the Ethereum Virtual Machine (EVM). Solidity lacks native support for decimal or fractional numbers, leading to truncation of division results to integers. This can result in inaccuracies or unexpected behaviors, especially when the numerator is not significantly larger than the denominator.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_1	XOX Staking.sol	L297 - L297	*Pending Fix*

Issue Type

LACK OF ZERO VALUE CHECK IN TOKEN TRANSFERS

S. No.	Severity	Detection Method	Instances
L001	Low	Automated	4

Description

The contract is found to lack a check for zero value transfers before executing the function. This oversight can lead t o failures when interacting with certain ERC20 tokens, such as LEND, which revert transactions when a zero transfer is attempted. This can disrupt the flow of the contract and lead to unexpected behavior.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_117	XOX Staking.sol	L191 - L191	*Pending Fix*
SSP_125012_117	XOX Staking.sol	L367 - L367	*Pending Fix*
SSP_125012_118	XOX Staking.sol	L346 - L346	*Pending Fix*
SSP_125012_119	XOX Staking.sol	L384 - L384	*Pending Fix*

Issue Type

MISSING EVENTS

S. No.	Severity	Detection Method	Instances
L002	Low	Automated	3

Description

Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the trans action's log—a special data structure in the blockchain.
These logs are associated with the address of the contract which can then be used by developers and auditors to ke ep track of the transactions.
The contract was found to be missing these events on the function which would make it difficult or impossible to trac k these transactions off-chain.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_14	XOX Staking.sol	L382 - L385	*Pending Fix*
SSP_125012_15	XOX Staking.sol	L406 - L406	*Pending Fix*
SSP_125012_16	XOX Staking.sol	L411 - L411	*Pending Fix*

Issue Type

OUTDATED COMPILER VERSION

S. No.	Severity	Detection Method	Instances
L003	Low	Automated	1

Description

Using an outdated compiler version can be problematic especially if there are publicly disclosed bugs and issues that affect the current compiler version.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_130	XOX Staking.sol	L2 - L2	*Pending Fix*

Issue Type

ADDING A RETURN STATEMENT WHEN THE FUNCTION DEFINES A NAMED RETURN VARIABLE IS REDUNDANT

S. No.	Severity	Detection Method	Instances
I001	Informational	Automated	1

Description

The contract defines a named return despite having a return statement. This causes redundancy and can lead to pot ential confusion in the code.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_120	XOX Staking.sol	L462 - L464	*Pending Fix*

Issue Type

BLOCK VALUES AS A PROXY FOR TIME

S. No.	Severity	Detection Method	Instances
I002	Informational	Automated	3

Description

Contracts often need access to time values to perform certain types of functionality. Values such as `block.timestamp` and `block.number` can be used to determine the current time or the time delta. However, they are not recommended for most use cases.

For `block.number`, as Ethereum block times are generally around 14 seconds, the delta between blocks can be predicted. The block times, on the other hand, do not remain constant and are subject to change for a number of reasons, e.g., fork reorganizations and the difficulty bomb.

Due to variable block times, `block.number` should not be relied on for precise calculations of time.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_132	XOX Staking.sol	L196 - L196	*Pending Fix*
SSP_125012_132	XOX Staking.sol	L236 - L236	*Pending Fix*
SSP_125012_133	XOX Staking.sol	L287 - L287	*Pending Fix*

Issue Type

CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES

S. No.	Severity	Detection Method	Instances
I003	Informational	Automated	3

Description

The contract defines time-related variables as `uint256`. Since Unix timestamps can be adequately represented usin g `uint48`, utilizing `uint256` for these variables results in unnecessary gas consumption due to larger storage require ments.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_121	XOX Staking.sol	L285 - L285	*Pending Fix*
SSP_125012_122	XOX Staking.sol	L294 - L294	*Pending Fix*
SSP_125012_123	XOX Staking.sol	L474 - L474	*Pending Fix*

Issue Type

CONTRACT NAME SHOULD USE PASCALCASE

S. No.	Severity	Detection Method	Instances
I004	Informational	Automated	1

Description

In Solidity, it's advisable to follow the established style guide, which dictates that contract names should be in PascalCase and match their respective file names. This adherence to consistent naming conventions not only enhances code readability and maintainability but also facilitates smoother collaboration among developers. By aligning with these recommendations, you make your code more accessible to others and reduce the likelihood of naming conflicts, contributing to a well-organized and efficient coding environment.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_125	XOX Staking.sol	L35 - L35	*Pending Fix*

Issue Type

MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

S. No.	Severity	Detection Method	Instances
I005	Informational	Automated	1

Description

Natspec `@author` tags are missing from contract declarations in the code. This reduces code clarity and makes it difficult to determine the original author or organization responsible for the contract. The absence of an `@author` tag can lead to challenges in tracking ownership, verifying authenticity, and maintaining proper documentation.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_9	XOX Staking.sol	L35 - L481	*Pending Fix*

Issue Type

MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION

S. No.	Severity	Detection Method	Instances
I006	Informational	Automated	1

Description

Natspec `@dev` tags are missing from contract declarations in the code. This reduces code clarity and makes it difficult for developers to understand the contract's internal logic, implementation details, and potential caveats.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_131	XOX Staking.sol	L35 - L481	*Pending Fix*

Issue Type

MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS

S. No.	Severity	Detection Method	Instances
I007	Informational	Automated	10

Description

Natspec `@dev` comments are missing from a function declaration in the code. The `@dev` tag provides a detailed explanation of a function's purpose, assumptions, and behavior. Lack of `@dev` documentation can: reduce code maintainability, making it harder for developers to understand the function's implementation details.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_51	XOX Staking.sol	L225 - L267	*Pending Fix*
SSP_125012_52	XOX Staking.sol	L281 - L319	*Pending Fix*
SSP_125012_53	XOX Staking.sol	L331 - L349	*Pending Fix*
SSP_125012_54	XOX Staking.sol	L363 - L373	*Pending Fix*
SSP_125012_55	XOX Staking.sol	L382 - L385	*Pending Fix*
SSP_125012_56	XOX Staking.sol	L397 - L401	*Pending Fix*
SSP_125012_57	XOX Staking.sol	L406 - L406	*Pending Fix*
SSP_125012_58	XOX Staking.sol	L411 - L411	*Pending Fix*
SSP_125012_59	XOX Staking.sol	L427 - L446	*Pending Fix*
SSP_125012_60	XOX Staking.sol	L462 - L464	*Pending Fix*

Issue Type

MISSING @INHERITDOC ON OVERRIDE FUNCTIONS

S. No.	Severity	Detection Method	Instances
I008	Informational	Automated	2

Description

Overridden functions in Solidity should include the `@inheritdoc` tag in their NatSpec comments to ensure documentation consistency and clarity. Failing to include `@inheritdoc` can lead to incomplete or missing documentation, making it difficult for developers and auditors to understand the function's intended behavior. This lack of clarity can increase maintenance overhead and potentially lead to misinterpretations of contract logic.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_11	XOX Staking.sol	L152 - L160	⚠️ *Pending Fix*
SSP_125012_12	XOX Staking.sol	L474 - L480	⚠️ *Pending Fix*

Issue Type

MISSING NATSPEC COMMENTS IN SCOPE BLOCKS

S. No.	Severity	Detection Method	Instances
I009	Informational	Automated	8

Description

NatSpec comments are missing from an unnamed scope block. Scope blocks can group computations or temporary l ogic and should be documented to clarify purpose and expected behavior. Lack of NatSpec documentation can redu ce readability, making it hard to understand why the scope block exists.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_109	XOX Staking.sol	L205 - L205	*Pending Fix*
SSP_125012_110	XOX Staking.sol	L242 - L257	*Pending Fix*
SSP_125012_111	XOX Staking.sol	L247 - L252	*Pending Fix*
SSP_125012_112	XOX Staking.sol	L252 - L256	*Pending Fix*
SSP_125012_113	XOX Staking.sol	L257 - L261	*Pending Fix*
SSP_125012_114	XOX Staking.sol	L308 - L311	*Pending Fix*
SSP_125012_115	XOX Staking.sol	L313 - L313	*Pending Fix*
SSP_125012_116	XOX Staking.sol	L343 - L343	*Pending Fix*

Issue Type

MISSING NATSPEC DESCRIPTIONS FOR PUBLIC VARIABLE DECLARATIONS

S. No.	Severity	Detection Method	Instances
I010	Informational	Automated	11

Description

Public variable declarations without NatSpec descriptions are found in the code. NatSpec comments improve readability and provide clear documentation for externally accessible variables. Without them, developers and users may struggle to understand the role and impact of these variables when interacting with the contract.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_93	XOX Staking.sol	L45 - L45	*Pending Fix*
SSP_125012_94	XOX Staking.sol	L46 - L46	*Pending Fix*
SSP_125012_95	XOX Staking.sol	L50 - L50	*Pending Fix*
SSP_125012_96	XOX Staking.sol	L54 - L54	*Pending Fix*
SSP_125012_97	XOX Staking.sol	L58 - L58	*Pending Fix*
SSP_125012_98	XOX Staking.sol	L59 - L59	*Pending Fix*
SSP_125012_99	XOX Staking.sol	L60 - L60	*Pending Fix*
SSP_125012_100	XOX Staking.sol	L64 - L64	*Pending Fix*
SSP_125012_101	XOX Staking.sol	L65 - L65	*Pending Fix*
SSP_125012_102	XOX Staking.sol	L66 - L66	*Pending Fix*
SSP_125012_103	XOX Staking.sol	L68 - L68	*Pending Fix*

Issue Type

MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS

S. No.	Severity	Detection Method	Instances
I011	Informational	Automated	1

Description

Natspec `@notice` comments are missing from a constructor declaration in the code. The `@notice` tag provides an o verview of what the constructor does, helping users and developers quickly understand its purpose.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_8	XOX Staking.sol	L152 - L160	*Pending Fix*

Issue Type

MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS

S. No.	Severity	Detection Method	Instances
I012	Informational	Automated	1

Description

Natspec `@notice` comments are missing from a function declaration in the code. The `@notice` tag provides an over view of what the function does, helping users and developers quickly understand its purpose.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_2	XOX Staking.sol	L474 - L480	*Pending Fix*

Issue Type

MISSING UNDERSCORE IN NAMING VARIABLES

S. No.	Severity	Detection Method	Instances
I013	Informational	Automated	16

Description

Solidity style guide suggests using underscores as the prefix for non-external functions and state variables (private o r internal) but the contract was not found to be following the same.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_65	XOX Staking.sol	L45 - L45	*Pending Fix*
SSP_125012_66	XOX Staking.sol	L46 - L46	*Pending Fix*
SSP_125012_67	XOX Staking.sol	L49 - L49	*Pending Fix*
SSP_125012_68	XOX Staking.sol	L50 - L50	*Pending Fix*
SSP_125012_69	XOX Staking.sol	L53 - L53	*Pending Fix*
SSP_125012_70	XOX Staking.sol	L54 - L54	*Pending Fix*
SSP_125012_71	XOX Staking.sol	L57 - L57	*Pending Fix*
SSP_125012_72	XOX Staking.sol	L58 - L58	*Pending Fix*
SSP_125012_73	XOX Staking.sol	L59 - L59	*Pending Fix*
SSP_125012_74	XOX Staking.sol	L60 - L60	*Pending Fix*
SSP_125012_75	XOX Staking.sol	L63 - L63	*Pending Fix*
SSP_125012_76	XOX Staking.sol	L64 - L64	*Pending Fix*
SSP_125012_77	XOX Staking.sol	L65 - L65	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_125012_78	XOX Staking.sol	L66 - L66	⚠️ *Pending Fix*
SSP_125012_79	XOX Staking.sol	L68 - L68	⚠️ *Pending Fix*
SSP_125012_80	XOX Staking.sol	L71 - L71	⚠️ *Pending Fix*

Issue Type

NAME MAPPING PARAMETERS

S. No.	Severity	Detection Method	Instances
I014	Informational	Automated	1

Description

After Solidity 0.8.18, a feature was introduced to name mapping parameters. This helps in defining a purpose for each mapping and makes the code more descriptive.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_13	XOX Staking.sol	L128 - L128	*Pending Fix*

Issue Type

REVERT STATEMENTS WITHIN EXTERNAL AND PUBLIC FUNCTIONS CAN BE USED TO PERFORM DOS ATTACKS

S. No.	Severity	Detection Method	Instances
I015	Informational	Automated	25

Description

In public and external functions, improper use of `revert` can be exploited for Denial of Service (DoS) attacks. An attacker can intentionally trigger these 'revert' conditions, causing legitimate transactions to consistently fail. For example, if a function relies on specific conditions from user input or contract state, an attacker could manipulate these to continually force reverts, blocking the function's execution.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_19	XOX Staking.sol	L178 - L178	*Pending Fix*
SSP_125012_19	XOX Staking.sol	L226 - L226	*Pending Fix*
SSP_125012_19	XOX Staking.sol	L282 - L282	*Pending Fix*
SSP_125012_19	XOX Staking.sol	L332 - L332	*Pending Fix*
SSP_125012_20	XOX Staking.sol	L181 - L181	*Pending Fix*
SSP_125012_21	XOX Staking.sol	L183 - L183	*Pending Fix*
SSP_125012_22	XOX Staking.sol	L184 - L184	*Pending Fix*
SSP_125012_23	XOX Staking.sol	L185 - L185	*Pending Fix*

Bug ID	File Location	Line No.	Action Taken
SSP_125012_24	XOX Staking.sol	L188 - L188	⚠️ *Pending Fix*
SSP_125012_25	XOX Staking.sol	L194 - L194	⚠️ *Pending Fix*
SSP_125012_25	XOX Staking.sol	L370 - L370	⚠️ *Pending Fix*
SSP_125012_26	XOX Staking.sol	L230 - L230	⚠️ *Pending Fix*
SSP_125012_27	XOX Staking.sol	L231 - L231	⚠️ *Pending Fix*
SSP_125012_28	XOX Staking.sol	L244 - L244	⚠️ *Pending Fix*
SSP_125012_29	XOX Staking.sol	L286 - L286	⚠️ *Pending Fix*
SSP_125012_30	XOX Staking.sol	L287 - L287	⚠️ *Pending Fix*
SSP_125012_31	XOX Staking.sol	L291 - L291	⚠️ *Pending Fix*
SSP_125012_32	XOX Staking.sol	L300 - L300	⚠️ *Pending Fix*
SSP_125012_32	XOX Staking.sol	L341 - L341	⚠️ *Pending Fix*
SSP_125012_33	XOX Staking.sol	L336 - L336	⚠️ *Pending Fix*
SSP_125012_34	XOX Staking.sol	L338 - L338	⚠️ *Pending Fix*
SSP_125012_35	XOX Staking.sol	L364 - L364	⚠️ *Pending Fix*
SSP_125012_36	XOX Staking.sol	L383 - L383	⚠️ *Pending Fix*
SSP_125012_37	XOX Staking.sol	L398 - L398	⚠️ *Pending Fix*
SSP_125012_38	XOX Staking.sol	L438 - L438	⚠️ *Pending Fix*

Issue Type

ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT

S. No.	Severity	Detection Method	Instances
G001	Gas	Automated	4

Description

The contract is found to contain a struct with multiple variables defined in it. When a struct is assigned in a single operation, Solidity may perform costly storage operations, which can be inefficient. This often results in increased gas costs due to multiple SLOAD and SSTORE operations happening at once

Bug ID	File Location	Line No.	Action Taken
SSP_125012_47	XOX Staking.sol	L156 - L156	*Pending Fix*
SSP_125012_48	XOX Staking.sol	L157 - L157	*Pending Fix*
SSP_125012_49	XOX Staking.sol	L158 - L158	*Pending Fix*
SSP_125012_50	XOX Staking.sol	L159 - L159	*Pending Fix*

Issue Type

AVOID RE-STORING VALUES

S. No.	Severity	Detection Method	Instances
G002	Gas	Automated	1

Description

The function is found to be allowing re-storing the value in the contract's state variable even when the old value is equal to the new value. This practice results in unnecessary gas consumption due to the `Gsreset` operation (2900 gas), which could be avoided. If the old value and the new value are the same, not updating the storage would avoid this cost and could instead incur a `Gcoldsload` (2100 gas) or a `Gwarmaccess` (100 gas), potentially saving gas.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_81	XOX Staking.sol	L397 - L401	⚠️ *Pending Fix*

Issue Type

AVOID ZERO-TO-ONE STORAGE WRITES

S. No.	Severity	Detection Method	Instances
G003	Gas	Automated	1

Description

Writing a storage variable from zero to a non-zero value costs 22,100 gas (20,000 for the write and 2,100 for cold access), making it one of the most expensive operations. This is why patterns like OpenZeppelin's `ReentrancyGuard` use `1` and `2` instead of `0` and `1`—to avoid the high cost of zero-to-non-zero writes. Non-zero to non-zero updates cost only 5,000 gas.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_124	XOX Staking.sol	L313 - L313	⚠️ *Pending Fix*

Issue Type

CACHE ADDRESS(THIS) WHEN USED MORE THAN ONCE

S. No.	Severity	Detection Method	Instances
G004	Gas	Automated	9

Description

The repeated usage of `address(this)` within the contract could result in increased gas costs due to multiple executions of the same computation, potentially impacting efficiency and overall transaction expenses.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_88	XOX Staking.sol	L190 - L190	*Pending Fix*
SSP_125012_88	XOX Staking.sol	L366 - L366	*Pending Fix*
SSP_125012_89	XOX Staking.sol	L191 - L191	*Pending Fix*
SSP_125012_89	XOX Staking.sol	L367 - L367	*Pending Fix*
SSP_125012_90	XOX Staking.sol	L192 - L192	*Pending Fix*
SSP_125012_90	XOX Staking.sol	L368 - L368	*Pending Fix*
SSP_125012_91	XOX Staking.sol	L299 - L299	*Pending Fix*
SSP_125012_91	XOX Staking.sol	L340 - L340	*Pending Fix*
SSP_125012_92	XOX Staking.sol	L453 - L453	*Pending Fix*

Issue Type

CHEAPER INEQUALITIES IN IF()

S. No.	Severity	Detection Method	Instances
G005	Gas	Automated	8

Description

The contract was found to be doing comparisons using inequalities inside the if statement.
When inside the `if` statements, non-strict inequalities `(>=, <=)` are usually cheaper than the strict equalities `(>, <)`.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_40	XOX Staking.sol	L184 - L184	*Pending Fix*
SSP_125012_41	XOX Staking.sol	L185 - L185	*Pending Fix*
SSP_125012_42	XOX Staking.sol	L287 - L287	*Pending Fix*
SSP_125012_43	XOX Staking.sol	L291 - L291	*Pending Fix*
SSP_125012_44	XOX Staking.sol	L300 - L300	*Pending Fix*
SSP_125012_44	XOX Staking.sol	L341 - L341	*Pending Fix*
SSP_125012_45	XOX Staking.sol	L305 - L305	*Pending Fix*
SSP_125012_46	XOX Staking.sol	L308 - L308	*Pending Fix*

Issue Type

DEFAULT INT VALUES ARE MANUALLY RESET

S. No.	Severity	Detection Method	Instances
G006	Gas	Automated	5

Description

The contract is found to inefficiently reset integer variables to their default value of zero using manual assignment. In Solidity, manually setting a variable to its default value does not free up storage space, leading to unnecessary gas c onsumption. Instead, using the `.delete` keyword can achieve the same result while also freeing up storage space on the Ethereum blockchain, resulting in gas cost savings.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_83	XOX Staking.sol	L201 - L201	*Pending Fix*
SSP_125012_84	XOX Staking.sol	L202 - L202	*Pending Fix*
SSP_125012_85	XOX Staking.sol	L203 - L203	*Pending Fix*
SSP_125012_86	XOX Staking.sol	L253 - L253	*Pending Fix*
SSP_125012_87	XOX Staking.sol	L258 - L258	*Pending Fix*

Issue Type

DEFINE CONSTRUCTOR AS PAYABLE

S. No.	Severity	Detection Method	Instances
G007	Gas	Automated	1

Description

Developers can save around 10 opcodes and some gas if the constructors are defined as payable.
However, it should be noted that it comes with risks because payable constructors can accept ETH during deployment.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_17	XOX Staking.sol	L152 - L160	*Pending Fix*

Issue Type

FUNCTIONS CAN BE IN-LINED

S. No.	Severity	Detection Method	Instances
G008	Gas	Automated	1

Description

The internal function was called only once throughout the contract. Internal functions cost more gas due to additiona l JUMP instructions and stack operations.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_82	XOX Staking.sol	L474 - L480	*Pending Fix*

Issue Type

REVERTING FUNCTIONS CAN BE PAYABLE

S. No.	Severity	Detection Method	Instances
G009	Gas	Automated	5

Description

If a function modifier such as `onlyOwner` is used, the function will revert if a normal user tries to pay the function. Marking the function as payable will lower the gas cost for legitimate callers because the compiler will not include checks for whether a payment was provided.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_3	XOX Staking.sol	L363 - L373	*Pending Fix*
SSP_125012_4	XOX Staking.sol	L382 - L385	*Pending Fix*
SSP_125012_5	XOX Staking.sol	L397 - L401	*Pending Fix*
SSP_125012_6	XOX Staking.sol	L406 - L406	*Pending Fix*
SSP_125012_7	XOX Staking.sol	L411 - L411	*Pending Fix*

Issue Type

FUNCTION SHOULD RETURN STRUCT

S. No.	Severity	Detection Method	Instances
G010	Gas	Automated	1

Description

The function was detected to be returning multiple values.
Consider using a `struct` instead of multiple return values for the function. It can improve code readability.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_18	XOX Staking.sol	L427 - L446	*Pending Fix*

Issue Type

GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION

S. No.	Severity	Detection Method	Instances
G011	Gas	Automated	1

Description

The contract is found to use multiple operands within a single `if` or `else if` statement, which can lead to unnecessary gas consumption due to the way the EVM evaluates compound boolean expressions. Each operand in a compound condition is evaluated even if the first condition fails, unless short-circuiting occurs, and the combined logic can result in more complex bytecode and higher gas usage compared to using nested `if` statements. This inefficiency is particularly relevant in functions that are called frequently or within loops.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_39	XOX Staking.sol	L291 - L291	⚠️ *Pending Fix*

Issue Type

SMALLER DATA TYPES COST MORE

S. No.	Severity	Detection Method	Instances
G012	Gas	Automated	1

Description

Usage of smaller integer types such as `uint8`, `uint16`, `int8`, or `int16` in arithmetic operations incur additional gas costs compared to the default `uint` and `int` types, which are typically `uint256` and `int256` respectively.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_10	XOX Staking.sol	L297 - L297	*Pending Fix*

Issue Type

STORAGE VARIABLE CACHING IN MEMORY

S. No.	Severity	Detection Method	Instances
G013	Gas	Automated	10

Description

The contract is using the state variable multiple times in the function.
SLOADs are expensive (100 gas after the 1st one) compared to MLOAD / MSTORE (3 gas each).

Bug ID	File Location	Line No.	Action Taken
SSP_125012_104	XOX Staking.sol	L177 - L208	*Pending Fix*
SSP_125012_104	XOX Staking.sol	L177 - L208	*Pending Fix*
SSP_125012_104	XOX Staking.sol	L177 - L208	*Pending Fix*
SSP_125012_105	XOX Staking.sol	L225 - L267	*Pending Fix*
SSP_125012_106	XOX Staking.sol	L281 - L319	*Pending Fix*
SSP_125012_106	XOX Staking.sol	L281 - L319	*Pending Fix*
SSP_125012_106	XOX Staking.sol	L281 - L319	*Pending Fix*
SSP_125012_107	XOX Staking.sol	L331 - L349	*Pending Fix*
SSP_125012_107	XOX Staking.sol	L331 - L349	*Pending Fix*
SSP_125012_108	XOX Staking.sol	L363 - L373	*Pending Fix*

Issue Type

STORING STORAGE VARIABLES IN MEMORY

S. No.	Severity	Detection Method	Instances
G014	Gas	Automated	5

Description

Whenever a struct, array, or a mapping is stored and copied to a memory variable, each member is read from the sto rage and then copied. This becomes expensive. This could easily be optimized by using the storage keyword which j ust stores a pointer to the storage instead, making the whole process a lot cheaper.

Bug ID	File Location	Line No.	Action Taken
SSP_125012_61	XOX Staking.sol	L187 - L187	*Pending Fix*
SSP_125012_62	XOX Staking.sol	L228 - L228	*Pending Fix*
SSP_125012_62	XOX Staking.sol	L334 - L334	*Pending Fix*
SSP_125012_63	XOX Staking.sol	L284 - L284	*Pending Fix*
SSP_125012_64	XOX Staking.sol	L439 - L439	*Pending Fix*

05. Scan History

● Critical ● High ● Medium ● Low ● Informational ● Gas

No	Date	Security Score	Scan Overview
1.	2026-02-24	**93.37**	● 0 ● 0 ● 1 ● 7 ● 79 ● 42

06. Disclaimer

The Reports neither endorse nor condemn any specific project or team, nor do they guarantee the security of any specific project. The contents of this report do not, and should not be interpreted as having any bearing on, the economics of tokens, token sales, or any other goods, services, or assets.

The security audit is not meant to replace functional testing done before a software release.

There is no warranty that all possible security issues of a particular smart contract(s) will be found by the tool, i.e., It is not guaranteed that there will not be any further findings based solely on the results of this evaluation.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. There is no warranty or representation made by this report to any Third Party in regards to the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business.

In no way should a third party use these reports to make any decisions about buying or selling a token, product, service, or any other asset. It should be noted that this report is not investment advice, is not intended to be relied on as investment advice, and has no endorsement of this project or team. It does not serve as a guarantee as to the project's absolute security.

The assessment provided by SolidityScan is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. SolidityScan owes no duty to any third party by virtue of publishing these Reports.

As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent manual audits including manual audit and a public bug bounty program to ensure the security of the smart contracts.